

CONTRIBUTING

Contributing to HelixCode Documentation

This guide explains how to update and maintain the HelixCode documentation.

Table of Contents

1. [Documentation Structure](#)
2. [Updating the Manual](#)
3. [Build Process Integration](#)
4. [Testing Documentation Locally](#)
5. [Automated Workflows](#)
6. [Best Practices](#)

Documentation Structure

```
HelixCode/
├── docs/                                # Source documentation (Markdown)
│   ├── USER_MANUAL.md                 # Comprehensive user manual
│   ├── USER_GUIDE.md                  # Quick start guide
│   ├── API_REFERENCE.md                # API documentation
│   ├── ARCHITECTURE.md                # System architecture
│   ├── CONTRIBUTING.md                # This file
│   └── ...
├── website/                            # Generated website files
│   ├── manual/                        # Manual in HTML format
│   │   ├── index.html
│   │   ├── USER_MANUAL.html
│   │   ├── USER_MANUAL.md
│   │   └── style.css
│   ├── guides/                        # Additional guides
│   └── pdf/                           # PDF versions (optional)
└── scripts/
    ├── sync-manual.sh                 # Sync manual to website
    ├── update-docs.sh                 # Master documentation script
    └── build.sh                       # Main build script
```

Updating the Manual

Quick Update

1. Edit the Markdown source:

```
# Edit the manual
vim docs/USER_MANUAL.md
```

2. Run the sync script:

```
# Single command to sync
make sync-manual

# Or use the script directly
./scripts/sync-manual.sh
```

Full Documentation Update

For a complete documentation update including HTML generation, PDF creation, and changelog:

```
# Basic update (Markdown to HTML + sync)
./scripts/update-docs.sh

# With PDF generation
./scripts/update-docs.sh --pdf

# With PDF and changelog update
./scripts/update-docs.sh --pdf --changelog

# Verbose output
./scripts/update-docs.sh --pdf --changelog --verbose
```

What Gets Updated

The documentation sync process:

1. **Timestamps:** Updates “Last Updated” dates in all Markdown files
2. **HTML Conversion:** Converts Markdown to HTML using pandoc
3. **Website Sync:** Copies files to the website/ directory
4. **Metadata:** Generates sync metadata and timestamps
5. **PDF (optional):** Creates PDF versions using pandoc
6. **Changelog (optional):** Adds entry to CHANGELOG.md

Build Process Integration

Makefile Targets

The build system provides several targets for documentation:

```
# Sync user manual to website
make sync-manual

# Convert Markdown to HTML only
make manual-html

# Build all documentation
make docs

# Full release build (code + docs + tests)
make release

# Generate all assets (logo + docs)
make assets
```

Build Script Integration

The main build script (`scripts/build.sh`) includes documentation by default:

```
# Standard build (includes docs)
./scripts/build.sh

# Build without documentation
./scripts/build.sh --no-docs

# Build with tests
./scripts/build.sh --with-tests

# Full release build
./scripts/build.sh --with-tests
```

Pre-Build Hook

Documentation is automatically regenerated before builds when using: `- make release - make assets - scripts/build.sh` (unless `--no-docs` is specified)

Testing Documentation Locally

Prerequisites

Install the required tools:

```
# macOS
brew install pandoc

# Linux (Debian/Ubuntu)
sudo apt-get install pandoc

# For PDF generation (optional)
brew install basicstex # macOS
sudo apt-get install texlive-xetex # Linux
```

Preview HTML Locally

1. Generate the HTML:

```
make manual-html
# or
./scripts/sync-manual.sh
```

2. Open in browser:

```
# macOS
open website/manual/index.html

# Linux
xdg-open website/manual/index.html

# Or navigate to:
file:///path/to/HelixCode/website/manual/index.html
```

Validate Markdown

Check for common Markdown issues:

```
# Check for broken links
grep -r "\[.*\](.*)" docs/*.md

# Validate Markdown syntax (if markdownlint is installed)
markdownlint docs/*.md
```

Test the Build Process

Test the complete build workflow:

```
# Test manual sync
make sync-manual
```

```
# Test full documentation build
make docs
```

```
# Test release build
make release
```

Automated Workflows

Pre-Commit Hook (Optional)

A pre-commit hook can automatically update documentation before commits. To install:

```
# Make the hook executable
chmod +x .git/hooks/pre-commit

# The hook will:
# 1. Detect if manual files changed
# 2. Regenerate HTML if needed
# 3. Update timestamps
# 4. Stage updated files
```

To skip the hook for a specific commit:

```
git commit --no-verify -m "Your message"
```

CI/CD Integration

For continuous integration, add to your CI pipeline:

```
# Example for GitHub Actions
- name: Build Documentation
  run: |
    make docs

- name: Check for Changes
  run: |
    git diff --exit-code website/
```

Best Practices

Writing Documentation

1. **Clear Structure:** Use hierarchical headings (H1 > H2 > H3)
2. **Code Examples:** Include practical examples with proper syntax highlighting
3. **Tables:** Use tables for comparison and configuration reference
4. **Links:** Use relative links for internal documentation
5. **Images:** Store images in docs/images/ and reference them relatively

Markdown Style

H1 - Main Title

H2 - Section

H3 - Subsection

****Bold for emphasis****

Italic for terms

``code`` for inline code

```
```language
code block
```
```

- Bullet lists

1. Numbered lists

| | |
|----------|----------|
| Column 1 | Column 2 |
| ----- | ----- |
| Value | Value |

Version Control

1. **Update Timestamps:** Always update “Last Updated” dates
2. **Commit Messages:** Use conventional commits

```
docs: update user manual with new LLM providers
docs: add troubleshooting section
docs: fix typos in API reference
```

3. **Review Changes:** Always review HTML output before committing

```
make sync-manual
open website/manual/USER_MANUAL.html
git diff website/
```

Documentation Checklist

Before committing documentation changes:

- ☐ Markdown source updated
- ☐ Timestamps updated (automated)
- ☐ HTML generated and reviewed
- ☐ Links tested and working
- ☐ Code examples tested
- ☐ Screenshots updated (if needed)
- ☐ CHANGELOG.md updated (for major changes)
- ☐ Version number incremented (if applicable)

Common Tasks

Adding a New Documentation File

1. Create the Markdown file in docs/:

```
touch docs/NEW_GUIDE.md
```

2. Add it to the update script:

```
# Edit scripts/update-docs.sh
# Add to MARKDOWN_FILES array
```

3. Update the sync script if needed:

```
# Edit scripts/sync-manual.sh
# Add conversion logic
```

Updating Styling

The CSS is located at `website/manual/style.css`. To update:

1. Edit the CSS in `scripts/sync-manual.sh` (it's generated)
2. Re-run the sync:

```
make sync-manual
```

3. Refresh your browser to see changes

Troubleshooting

Problem: HTML not updating

```
# Clear website directory and regenerate
rm -rf website/manual/*
make sync-manual
```

Problem: Timestamps not updating

```
# Run update script explicitly
./scripts/update-docs.sh
```

Problem: pandoc errors

```
# Check pandoc version
pandoc --version
```

```
# Reinstall if needed
brew reinstall pandoc # macOS
```

Workflow Summary

For Quick Updates

1. Edit `docs/USER_MANUAL.md`
2. `make sync-manual`
3. `git add docs/ website/`
4. `git commit -m "docs: update manual"`
5. `git push`

For Release Documentation

1. Edit docs/*.md files
2. ./scripts/update-docs.sh --pdf --changelog
3. Review changes in website/
4. make test # Verify builds work
5. git add docs/ website/ CHANGELOG.md
6. git commit -m "docs: release documentation update"
7. git push

For Full Release

1. Update documentation
2. make release # Builds everything
3. Review bin/ and website/
4. git add .
5. git commit -m "release: version X.Y.Z"
6. git tag vX.Y.Z
7. git push --tags

Getting Help

- **Questions:** Open an issue on GitHub
- **Bugs:** Report documentation bugs in the issue tracker
- **Improvements:** Submit a pull request with your changes

Additional Resources

- [Markdown Guide](#)
- [Pandoc Manual](#)
- [HelixCode Documentation](#)

Last Updated: 2025-11-06 **Version:** 1.0